CourseListModel — Explained

A walkthrough of the AEM Sling Model that powers the course list component

Big Picture

`CourseListModel` is a Sling Model in AEM. Its job is to power a "course list" component on a page: it goes into the DAM, finds all the Course Content Fragments under a configured root, sorts them by start date, trims them to a limit, and exposes them to HTL (the template language) along with a list of category filter buttons.

Think of it as: *"Component on page asks: give me courses to render. This class answers that question."*

Fields at the Top

These are the inputs the model needs to do its work:

- `request` — the current Sling request, used to get a `ResourceResolver`.
- `limit` — comes from the component's dialog. The author can say "show me 6 courses."
- `filterCategory` — also from the dialog (looks unused in the current logic, but it's wired up).
- `headingText` — heading text the author types in the dialog.
- `courseConfigService` — an OSGi service that holds global config like the DAM root path and a default limit. Keeps hardcoded paths out of the code.
- `courses` and `categoryFilters` — the two lists HTL will actually render.

`init()` — The Orchestrator

This runs once after the model is constructed (`@PostConstruct`). It's basically a 4-step recipe:

- **Step 1.** Find the root folder in the DAM using the path from `CourseConfigService`. If it doesn't exist, log a warning and bail out.
- **Step 2.** Recursively walk that folder with `collectCourses(...)`, gathering every Course content fragment into one flat list.
- **Step 3.** Sort all collected courses by start date (earliest first), then trim to the effective limit (dialog value wins; otherwise use the service default).
- **Step 4.** Build the category filter list — walk the final course list and collect each unique category (id + title) in the order they appear. Uses a `HashSet` to dedupe and a `List` to preserve order.

`collectCourses(...)` — The Tree Walker

A classic recursive DAM traversal. For each child of the current folder:

- If it's an **asset**, try to adapt it to `ContentFragment`. If that works AND it's a Course fragment (checked by `isCourseFragment`), build a bean and add it.
- If it's a **folder** (`sling:Folder`, `sling:OrderedFolder`, or `nt:folder`), recurse into it.
- Anything else gets ignored.

`isCourseFragment(...)` — The Type Check

Content Fragments store their model path in `jcr:content/data/cq:model`. This method just reads that property and checks if it matches the known Course model path constant. That's how you tell a Course fragment apart from, say, a Person or Event fragment sitting in the same folder.

`buildBean(...)` — The Mapper

This is where a raw Content Fragment gets turned into a clean `CourseBean` that HTL can read. It does five things:

- **Title and description** — straight string reads.
- **Start date** — read as a `Calendar`, format it as "01 Jul 2026" for display, AND store the raw millis so step 3 of `init()` can sort numerically. If there's no start date, set millis to `Long.MAX_VALUE` so undated courses sort to the end.
- **End date** — same pattern, but no sort key needed.
- **Category normalization** — this is the messy part. The `courseCategory` field might come back as `course-category:engineering/engineering` or `tag:foo/bar` or just `engineering`. The code takes only the first value if it's comma-separated, strips a `tag:` prefix if present, and grabs the last segment after the final `/` or `:` — that's the leaf tag name.
- **Category title** — tries to resolve the original tag string via `TagManager` to get a proper human-readable title (e.g. "Engineering & Design"). If that fails, falls back to capitalizing the leaf name.

The Little Helpers

- `readString` / `readCalendar` — null-safe reads from a Content Fragment element. Without these, you'd get NPEs every time a field was empty.

- `formatDate` — wraps `SimpleDateFormat` with the chosen pattern and locale.
- `capitalize` — `engineering` → `Engineering`. Fallback when TagManager can't resolve a title.

The Getters at the Bottom

These are what HTL calls. `getCourses()`, `getCategoryFilters()`, `getHeadingText()`, `isEmpty()`, `getTotalCount()` — each one maps to something the template needs to render.

`CategoryFilter` — The Inner Class

A tiny POJO with `id` and `title`. The comment explains why it exists: HTL doesn't deal well with `Map.Entry`, so instead of exposing a `Map<String,String>`, the code exposes a `List<CategoryFilter>`. Cleaner template code, no surprises.

Data Flow at a Glance

Request comes in → `ResourceResolver` → DAM walk → bean list → sort & trim → category filter list → HTL renders the component.